

Avance Fase 3 — Semana 2 (Formativa)

Grupo 4 · MCDI500

Este notebook parte del conjunto ya limpio y validado en la Fase 2 (`data/processed/ens_procesado.csv`). No se rehace la limpieza ni se agregan datos nuevos: el objetivo de esta fase es reorganizar el código que ya funciona, medir su eficiencia, y decidir si el proyecto justifica recursividad.

```
In [1]: import sys
import time
from pathlib import Path

def _encontrar_raiz_temporal(marcador=".git"):
    actual = Path.cwd()
    for carpeta in [actual, *actual.parents]:
        if (carpeta / marcador).exists():
            return carpeta
    raise FileNotFoundError(f"No se encontro '{marcador}' en ningun directorio")

RAIZ = _encontrar_raiz_temporal()
sys.path.append(str(RAIZ / "src"))

from carga import encontrar_raiz_proyecto, cargar_datos, preparar, ejecutar

SEMILLA = 2026

import pandas as pd
print("Python:", sys.version.split()[0])
print("pandas:", pd.__version__)
print("Raiz del proyecto:", RAIZ.name)
```

Python: 3.13.7

pandas: 3.0.5

Raiz del proyecto: proyecto-grupo4-mcdi500

1. Carga del conjunto de la Fase 2

Se carga el archivo ya procesado y validado en la Fase 2, sin rehacer limpieza ni transformación alguna.

```
In [2]: ARCHIVO = RAIZ / "data" / "processed" / "ens_procesado.csv"
df = ejecutar(ARCHIVO)
df.head()
```

Cargado: 5511 filas y 23 columnas

Preparado: 24 columnas (incluye clave_búsqueda)

Out [2]:

	Edad	HTA	IMC	anos_estudio_MINSAL_1	GPAQ	as27	as28	Fexp
0	-1.256904	0	-1.031300	15.0	3	-0.433713	3.0	
1	-1.516452	0	-0.490394	14.0	3	NaN	NaN	
2	1.857664	1	-0.935179	15.0	1	1.339974	10.0	
3	-0.997357	0	-0.704314	16.0	3	NaN	NaN	
4	0.196561	0	0.604864	14.0	1	NaN	NaN	

5 rows × 24 columns

El conjunto antes de tocarlo

Antes de escribir cualquier función, revisamos brevemente el estado del conjunto que ya dejó lista la Fase 2. No se modifica nada aquí: es solo el punto de partida para las mediciones de esta fase.

In [3]:

```
print("Dimensiones:", df.shape)
print("\nTipos de datos:")
print(df.dtypes.value_counts())
print("\nValores nulos totales:", int(df.isnull().sum().sum()))
print("\nColumnas con nulos declarados (GPAQ imputada, as27/as28 con NaN genuino):")
print(df.isnull().sum()[df.isnull().sum() > 0])
```

Dimensiones: (5511, 24)

Tipos de datos:

int64 18

float64 6

Name: count, dtype: int64

Valores nulos totales: 1632

Columnas con nulos declarados (GPAQ imputada, as27/as28 con NaN genuino):

as27 816

as28 816

dtype: int64

2. Transformaciones aplicadas (apartado III)

No se aplica ninguna transformación de limpieza, imputación ni escalamiento adicional a las ya validadas en la Fase 2. La única preparación de este avance es agregar la columna `clave_búsqueda` (dentro de `preparar()`, en `src/carga.py`), necesaria porque `IdEncuesta` fue excluida del conjunto procesado y se requiere un identificador real para medir búsquedas, no un simple acceso por posición.

3. Medición de eficiencia (primera comparación): acceso por posición

Esta primera comparación mide **acceso a una posición ya conocida** (recorrer hasta la posición N frente a `df.iloc[N]`), no una búsqueda: si ya se sabe dónde está el dato, no hay nada que buscar. Es una comparación válida sobre la sobrecarga de `iterrows()`, pero no equivale a comparar estrategias de búsqueda. Esa comparación distinta se desarrolla en la sección 3.5.

```
In [4]: import sys
sys.path.append(str(RAIZ / "src"))

from medicion import buscar_recorriendo, buscar_por_indice, medir_tiempo, me

print("Funciones importadas correctamente desde src/medicion.py")

posicion_prueba = df.shape[0] - 1

r_a = buscar_recorriendo(df, posicion_prueba)
r_b = buscar_por_indice(df, posicion_prueba)

print("Encontrado por recorrido:", r_a is not None)
print("Encontrado por indice:", r_b is not None)
```

```
Funciones importadas correctamente desde src/medicion.py
Encontrado por recorrido: True
Encontrado por indice: True
```

3.1 Medición: tiempo de cada versión

Aplicamos las tres reglas para que la medición valga: repetir y conservar el mínimo, medir sobre el tamaño real del conjunto (5.511 filas), y comprobar que ambas versiones entregan el mismo resultado antes de comparar los tiempos.

```
In [5]: # posicion_prueba se define aqui mismo, sin depender de celdas anteriores
posicion_prueba = df.shape[0] - 1

tiempo_a, resultado_a = medir_tiempo(buscar_recorriendo, df, posicion_prueba)
tiempo_b, resultado_b = medir_tiempo(buscar_por_indice, df, posicion_prueba)

print(f"Recorriendo:  {tiempo_a:.6f} s")
print(f"Por indice:    {tiempo_b:.6f} s")
print(f"El acceso por indice es {tiempo_a / tiempo_b:.1f} veces mas rapido")

assert resultado_a.equals(resultado_b), "Las dos versiones no coinciden"
print("\nVerificado: ambas versiones entregan exactamente el mismo resultado")
```

```
Recorriendo: 0.029845 s
Por indice: 0.000015 s
El acceso por indice es 1935.9 veces mas rapido
```

Verificado: ambas versiones entregan exactamente el mismo resultado.

```
Por indice: 0.000015 s
El acceso por indice es 1935.9 veces mas rapido
```

Verificado: ambas versiones entregan exactamente el mismo resultado.

3.2 Interpretación

El acceso por posición (`buscar_por_indice`) resultó considerablemente más rápido que recorrer el conjunto fila por fila (`buscar_recorriendo`), midiendo sobre el peor caso posible (la última fila del conjunto). La diferencia se explica porque `buscar_recorriendo` recorre hasta 5.511 filas en el peor caso, mientras que `buscar_por_indice` accede directamente a la posición de memoria correspondiente.

Como se explicó al inicio de esta sección, esta comparación mide acceso a una posición ya conocida, no una búsqueda real. La decisión sobre qué estructura adoptar para el proyecto se desarrolla en la sección 3.5, con una comparación que sí responde esa pregunta.

3.3 Medición de memoria

El descriptor pide analizar complejidad temporal **y** espacial. El acceso por índice suele ganar en tiempo, pero implica mantener una estructura adicional en memoria. Medimos ambos aspectos con `tracemalloc` , siguiendo la sugerencia de la retroalimentación recibida.

```
In [6]: memoria_recorrido, _ = medir_memoria(buscar_recorriendo, df, posicion_prueba)
        memoria_indice, _ = medir_memoria(buscar_por_indice, df, posicion_prueba)

        print(f"Memoria maxima - recorrido: {memoria_recorrido / 1024:.2f} KB")
        print(f"Memoria maxima - por indice: {memoria_indice / 1024:.2f} KB")
```

```
Memoria maxima - recorrido: 1039.68 KB
Memoria maxima - por indice: 2.29 KB
```

3.3.1 Interpretación de la memoria

Contrario a lo esperado en el caso general (donde construir un índice suele costar memoria adicional), en esta comparación el acceso por índice también resultó más eficiente en memoria: 2,29 KB frente a 1039 KB del recorrido.

La explicación está en cómo opera cada función: `buscar_recorriendo` usa `iterrows()`, que construye una nueva `Series` de pandas en cada iteración del bucle -consumiendo memoria temporal en cada paso, aunque se descarte después-, mientras que `buscar_por_indice` con `.iloc` accede directamente a la posición en memoria ya existente del DataFrame, sin crear estructuras intermedias.

Esto no contradice el principio general (que construir un índice formal, como con `.set_index()`, sí tiene un costo de memoria), sino que muestra que **el costo real depende de la implementación específica**, no solo del enfoque conceptual. Por eso es imprescindible medir en el caso concreto, y no asumir el resultado de memoria.

3.3.2 Verificación: ¿el resultado es consistente?

Antes de confiar en una sola medición, la repetimos varias veces para confirmar que la diferencia de memoria no es casualidad.

```
In [7]: resultados_recorrido = []
resultados_indice = []

for _ in range(5):
    mem_r, _ = medir_memoria(buscar_recorriendo, df, posicion_prueba)
    mem_i, _ = medir_memoria(buscar_por_indice, df, posicion_prueba)
    resultados_recorrido.append(mem_r)
    resultados_indice.append(mem_i)

print("Memoria recorrido (5 mediciones, en KB):", [round(m/1024, 2) for m in resultados_recorrido])
print("Memoria indice (5 mediciones, en KB):", [round(m/1024, 2) for m in resultados_indice])

print(f"\nRecorrido - minimo: {min(resultados_recorrido)/1024:.2f} KB, maximo: {max(resultados_recorrido)/1024:.2f} KB")
print(f"Indice - minimo: {min(resultados_indice)/1024:.2f} KB, maximo: {max(resultados_indice)/1024:.2f} KB")
```

Memoria recorrido (5 mediciones, en KB): [1039.22, 1037.68, 1037.46, 1037.46, 1037.46]

Memoria indice (5 mediciones, en KB): [2.1, 2.1, 2.1, 2.1, 2.1]

Recorrido - minimo: 1037.46 KB, maximo: 1039.22 KB

Indice - minimo: 2.10 KB, maximo: 2.10 KB

3.3.3 Conclusión de la verificación

Las 5 repeticiones confirman que el resultado es estable: la memoria usada por `buscar_recorriendo` se mantiene siempre entre 1.037 y 1.039 KB, y la de `buscar_por_indice` es constante en 2,10 KB. La diferencia (~494 veces menos memoria con el índice) no es un dato aislado, sino un patrón reproducible atribuible a la diferencia estructural entre `iterrows()` y `.iloc` explicada arriba.

3.4 Sobre el punto de equilibrio en esta comparación

Como esta sección mide acceso por posición y no una búsqueda real, `.iloc` no construye ninguna estructura previa, por lo que no existe aquí un punto de equilibrio que calcular. El punto de equilibrio genuino se calcula en la sección 3.5, donde sí se compara contra una estructura que debe construirse antes de usarse.

3.5 Segunda comparación: búsqueda real por clave

A diferencia de la sección 3, aquí comparamos formas de resolver un problema de **búsqueda genuino**: encontrar la fila cuya clave es un valor determinado, sin saber de antemano dónde está.

```
In [8]: from medicion import (
        buscar_iterrows_por_clave, buscar_vectorizado,
        construir_indice_pandas, construir_indice_dict,
        buscar_indexado_pandas, buscar_indexado_dict,
        )

valor_buscado = df["clave_busqueda"].iloc[-1]

t_iterrows, r_iterrows = medir_tiempo(buscar_iterrows_por_clave, df, "clave_
t_vectorizado, r_vectorizado = medir_tiempo(buscar_vectorizado, df, "clave_b

t_construccion_pandas, df_indexado = medir_tiempo(construir_indice_pandas, c
t_indexado_pandas, r_indexado_pandas = medir_tiempo(buscar_indexado_pandas,

t_construccion_dict, indice_dict = medir_tiempo(construir_indice_dict, df, "

# Una búsqueda en diccionario es demasiado rápida para medirla sola:
# se repite N veces y se divide para obtener el tiempo por búsqueda.
N = 100000
inicio = time.perf_counter()
for _ in range(N):
    buscar_indexado_dict(indice_dict, valor_buscado)
t_indexado_dict = (time.perf_counter() - inicio) / N
r_indexado_dict = buscar_indexado_dict(indice_dict, valor_buscado)

print("=== Costo de busqueda (una vez construido, si aplica) ===")
print(f"Iterrows:           {t_iterrows:.6f} s")
print(f"Vectorizado:         {t_vectorizado:.6f} s")
print(f"Indexado (pandas):     {t_indexado_pandas:.6f} s")
print(f"Indexado (dict):      {t_indexado_dict:.9f} s")
print()
print("=== Costo de construccion (pago unico) ===")
print(f"set_index (pandas):    {t_construccion_pandas:.6f} s")
print(f"diccionario (nativo):   {t_construccion_dict:.6f} s")

assert (r_iterrows["clave_busqueda"] == r_vectorizado["clave_busqueda"]
        == r_indexado_pandas.name == r_indexado_dict["clave_busqueda"]), \
    "Las cuatro versiones no coinciden"
print("\nVerificado: las cuatro versiones entregan el mismo resultado.")
```

```
=== Costo de busqueda (una vez construido, si aplica) ===
Iterrows:          0.035714 s
Vectorizado:       0.000088 s
Indexado (pandas): 0.000014 s
Indexado (dict):   0.000000095 s
```

```
=== Costo de construccion (pago unico) ===
set_index (pandas): 0.000067 s
diccionario (nativo): 0.038395 s
```

Verificado: las cuatro versiones entregan el mismo resultado.

Punto de equilibrio real

El siguiente bloque calcula cuántas búsquedas hacen falta para que construir un índice se pague. Para cada estructura (`set_index` y `diccionario`) divide su costo de construcción por lo que ahorra cada búsqueda frente a la versión vectorizada. Si el proyecto va a hacer más búsquedas que ese número, conviene construir el índice.

```
In [9]: for nombre, t_construccion, t_indexado in [
        ("set_index (pandas)", t_construccion_pandas, t_indexado_pandas),
        ("diccionario (nativo)", t_construccion_dict, t_indexado_dict),
        ]:
        diferencia = t_vectorizado - t_indexado
        equilibrio = t_construccion / diferencia
        print(f"{nombre}: construir cuesta {t_construccion:.6f}s -> "
              f"compensa a partir de {equilibrio:.1f} busquedas")
```

```
set_index (pandas): construir cuesta 0.000067s -> compensa a partir de 0.9 b
usquedas
diccionario (nativo): construir cuesta 0.038395s -> compensa a partir de 43
7.6 busquedas
```

3.5.1 Conclusión de la segunda comparación

Separar la búsqueda real (esta sección) del acceso por posición (sección 3) muestra que gran parte de la diferencia original se debía a la sobrecarga de `iterrows()`, no a la estrategia de búsqueda en sí: la versión vectorizada, aun siendo $O(n)$, ya es unas 400 veces más rápida que el recorrido con `iterrows()`.

El punto de equilibrio real existe entre la búsqueda vectorizada y la indexada, porque solo la indexación paga un costo de construcción inicial. `set_index` de pandas compensa su costo prácticamente desde la primera búsqueda (en esta ejecución, menos de una), mientras que un diccionario nativo, aunque ofrece la búsqueda más rápida de las cuatro, requiere entre 430 y 450 búsquedas para justificar el costo de construirlo recorriendo el DataFrame con `iterrows()`. Esto muestra que la mejor estructura no depende solo de la velocidad final, sino también de cuántas veces se va a usar.

Decisión adoptada: se adopta `set_index` de pandas para cualquier búsqueda repetida sobre el conjunto. Aunque el diccionario nativo ofrece la búsqueda más rápida de las cuatro alternativas, su costo de construcción (entre 430 y 450 búsquedas para compensarse) lo hace menos práctico que `set_index`, que se paga desde la primera búsqueda. Como el proyecto necesita ubicar registros repetidamente al validar cruces entre variables como `as27` y `as28`, `set_index` es la opción más conveniente.

4. ¿Necesita este proyecto recursividad?

Antes de forzar una recursión artificial, evaluamos si el pipeline de la Fase 2 tiene un problema que la justifique.

```
In [10]: # Revision de los pasos del pipeline de Fase 2:
pasos_pipeline = [
    "seleccionar_variables_ens",
    "filtrar_ponderador_valido",
    "explorar_dataframe",
    "revisar_codigos_especiales",
    "marcar_codigos_no_respuesta",
    "imputar_nulos_numericos / imputar_nulos_categoricos",
    "codificar_one_hot",
    "escalar_caracteristicas",
    "validar_dataset",
]

print(f"El pipeline tiene {len(pasos_pipeline)} pasos, en una secuencia FIJA")
for i, paso in enumerate(pasos_pipeline, 1):
    print(f"  {i}. {paso}")
```

El pipeline tiene 9 pasos, en una secuencia FIJA y conocida:

1. seleccionar_variables_ens
2. filtrar_ponderador_valido
3. explorar_dataframe
4. revisar_codigos_especiales
5. marcar_codigos_no_respuesta
6. imputar_nulos_numericos / imputar_nulos_categoricos
7. codificar_one_hot
8. escalar_caracteristicas
9. validar_dataset

4.1 Conclusión: no se justifica recursión en el pipeline principal

El pipeline principal tiene una secuencia fija y conocida de nueve pasos, sin niveles de profundidad variable ni una estructura autosimilar. Un enfoque iterativo (aplicar cada función en orden) es preferible: es más simple, más legible, y no arriesga agotar la pila de llamadas de Python.

La única función recursiva presente en el proyecto es `aplanar()`, utilizada en la Fase 1 para convertir el diccionario anidado de metadatos del proyecto en pares clave-valor. Esa función sí se justifica, porque la profundidad de anidamiento del diccionario no se conoce de antemano al escribir el código: podría tener uno, dos o más niveles, y la recursión se adapta a cualquiera de ellos sin necesidad de escribir un bucle distinto para cada caso.

5. Referencia al foro técnico

En el foro técnico de la semana comentamos que una comparación de eficiencia puede parecer válida aunque esté midiendo algo distinto de lo que se cree, y que el tamaño de una diferencia (por ejemplo, "2.000 veces más rápido") no garantiza por sí solo que la comparación esté bien diseñada.

A partir de esa discusión, revisamos nuestra propia comparación inicial (`iterrows()` frente a `.iloc`) y encontramos que mezclaba acceso a una posición conocida con búsqueda real, además de incluir la sobrecarga propia de `iterrows()`. Aplicamos lo discutido separando ambos problemas en las secciones 3 y 3.5 de este notebook, lo que nos permitió calcular un punto de equilibrio real y tomar una decisión de diseño fundamentada.